REVISION CHEAT SHEET:

PATTERNS / C#

Pattern Matching and Modern C#

Interview-ready notes · examples · C# · diagrams

Topic 26 of 40

Pattern matching is the C# feature set used to answer questions such as:

EXAMPLE / FLOW

```
What type is this value?

Is it null?

Does it equal a particular constant?

Is it inside a numeric range?

Do certain properties have particular values?

Can this object be deconstructed into a particular shape?

Does this sequence have a particular structure?

Which combination of conditions describes this state?
```

Instead of repeatedly writing casts, nested `if` statements, and temporary variables, patterns let you describe the **shape or state you expect**.

C# supports patterns primarily in three locations:

C#

```
value is pattern

switch (value)
{
    case pattern:
}

value switch
{
    pattern => result
}
```

Current C# includes declaration, type, constant, `var`, property, positional, relational, logical, parenthesized, list, slice, and discard patterns.

1. Why Pattern Matching Exists

Consider old-style type checking:

C#

```
if (value is Gift)
{
    Gift gift = (Gift)value;

    Console.WriteLine(
```

```
        gift.TotalAmount);
}
```

PATTERNS / C#

The code performs:

```
EXAMPLE / FLOW
Type test
    ↓
Cast
    ↓
Create variable
    ↓
Use object
```

Pattern matching combines these operations:

```
C#
if (value is Gift gift)
{
    Console.WriteLine(
        gift.TotalAmount);
}
```

Now:

```
EXAMPLE / FLOW
Test runtime type
+
Safely capture typed value
```

happen together.

2. Pattern Matching Mental Model

A pattern answers:

> **Does this value have the type, value, or shape described by this pattern?**

For example:

```
C#
gift is
{
    Status: GiftStatus.Completed,
    TotalAmount: >= 1000m
}
```

describes:

```
EXAMPLE / FLOW
Gift is non-null

AND

Status is Completed

AND

TotalAmount is at least 1000
```

The code describes the state rather than manually extracting every property.

PATTERNS / C#

3. `is` Pattern Expressions

The simplest pattern-matching form uses:

```C#
is
```

General shape:

```C#
expression is pattern
```

Example:

```C#
if (value is Gift)
{
}
```

This asks:

```EXAMPLE / FLOW
Is the runtime value compatible
with Gift?
```

The `is` operator supports runtime type tests, null patterns, and the broader C# pattern system.

4. Type Pattern

If you only need to test the type:

```C#
if (value is Gift)
{
    Console.WriteLine(
        "Value is a Gift.");
}
```

No variable is created.

Use this when:

```EXAMPLE / FLOW
The type itself matters

but

you don't need the typed value afterward
```

5. Declaration Pattern

A declaration pattern tests the runtime type **and** introduces a variable.

```C#
if (value is Gift gift)
{
    Console.WriteLine(
```

```
        gift.TotalAmount);
}
```

PATTERNS / C#

Here:

```
EXAMPLE / FLOW
value
    →
Pattern input

Gift
    →
Runtime type requirement

gift
    →
New typed variable
```

The declaration pattern only succeeds for a non-null runtime value compatible with the specified type.

Declaration Pattern Replaces Test + Cast

Old:

```
C#
if (value is Gift)
{
    Gift gift = (Gift)value;
}
```

Modern:

```
C#
if (value is Gift gift)
{
}
```

Prefer the second form.

6. Pattern Variable and Definite Assignment

Consider:

```
C#
if (value is Gift gift)
{
    Console.WriteLine(
        gift.TotalAmount);
}
```

Inside the block, the compiler knows:

```
EXAMPLE / FLOW
Pattern succeeded
    ↓
gift definitely contains a Gift
```

You don't need:

```C#
Gift? gift
```

or another null check.

The compiler tracks this through its definite-assignment analysis.

7. Pattern Variables with &&

Pattern variables can often be used later in the same Boolean condition when control flow proves the match succeeded.

```C#
if (
    value is Gift gift
    &&
    gift.TotalAmount >= 1000m)
{
    Console.WriteLine(
        gift.GiftId);
}
```

Evaluation is:

```EXAMPLE / FLOW
value is Gift gift
        ↓
false?
    →
stop
```

If true:

```EXAMPLE / FLOW
gift is definitely assigned
        ↓
evaluate amount condition
```

8. Constant Pattern

A constant pattern checks whether the input equals a particular constant.

Example:

```C#
if (
    gift.Status
    is GiftStatus.Completed)
{
}
```

Another:

```C#
if (
    status
    is GiftStatus.Cancelled)
{
}
```

Constant patterns are particularly useful inside:

PATTERNS / C#

EXAMPLE / FLOW

```
switch expressions

logical patterns

nested property patterns
```

9. Null Pattern

One of the most important constant-pattern uses is:

C#

```
if (gift is null)
{
}
```

For non-null:

C#

```
if (gift is not null)
{
}
```

Microsoft recommends `is null` / `is not null` as reliable null checks because those pattern tests aren't affected by user-overloaded `==` or `!=` operators.

Why This Can Be Better Than == null

A custom class could overload:

C#

```
==
```

Therefore:

C#

```
value == null
```

might involve custom equality behavior.

But:

C#

```
value is null
```

means:

EXAMPLE / FLOW

```
Actual null pattern test
```

without invoking an overloaded equality operator.

Recommended Null Pattern

```C#
if (gift is not null)
{
    Process(
        gift);
}
```

This also helps nullable-reference analysis understand that `gift` is non-null inside the branch.

10. var Pattern

The `var` pattern always matches and captures the evaluated value.

Example:

```C#
if (
    CalculateAllocatedAmount(gift)
        is var allocated)
{
    Console.WriteLine(
        allocated);
}
```

Unlike a declaration pattern:

```C#
value is Gift gift
```

the `var` pattern isn't checking a runtime type.

It is essentially saying:

> **Evaluate this expression and give the result a name.**

Microsoft documents that a `var` pattern matches every value, including `null`.

Example with a Guard

```C#
string GetAllocationStatus(
    Gift gift)
    =>
    gift switch
    {
        _
            when gift.Allocations.Sum(
                a => a.Amount)
                is var total
                &&
                total == gift.TotalAmount
            =>
            $"Fully allocated: {total:C}",

        _
            =>
```

```
            "Not fully allocated"
    };
```

PATTERNS / C#

In practice, don't use `var` patterns merely to make simple code look sophisticated.

Use them when capturing an intermediate expression genuinely improves readability.

11. Discard Pattern _

The discard pattern:

C#

```
_
```

matches anything.

Example:

C#

```
string Describe(
    GiftStatus status)
    =>
    status switch
    {
        GiftStatus.Pending
            =>
            "Pending",

        GiftStatus.Completed
            =>
            "Completed",


        _
            =>
            "Other"
    };
```

The final:

C#

```
_
```

means:

EXAMPLE / FLOW

```
Everything not handled above
```

The discard pattern can also match `null`.

12. Switch Statement with Patterns

Traditional `switch`:

C#

```
switch (value)
{
    case Gift gift:
        Console.WriteLine(
            gift.TotalAmount);
        break;
```

```
    case Campaign campaign:
        Console.WriteLine(
            campaign.Name);
        break;

    case null:
        Console.WriteLine(
            "No value.");
        break;

    default:
        Console.WriteLine(
            "Unknown.");
        break;
}
```

This is useful when each branch performs:

EXAMPLE / FLOW

```
Statements

Side effects

Multiple operations
```

13. Switch Expression

Modern C# also supports:

C#

```
expression switch
{
    pattern => result,
    pattern => result,
    _       => result
}
```

Example:

C#

```
string description =
    status switch
    {
        GiftStatus.Pending
            =>
            "Awaiting processing",

        GiftStatus.Completed
            =>
            "Completed",

        GiftStatus.Cancelled
            =>
            "Cancelled",


        _
            =>
            "Unknown"
    };
```

A switch expression returns the expression from the **first arm whose pattern matches and whose optional guard succeeds**. Arms are tested in textual order.

PATTERNS / C#

Switch Statement vs Switch Expression

Use a switch statement when branches primarily:

EXAMPLE / FLOW

```
Perform actions
```

Use a switch expression when branches primarily:

EXAMPLE / FLOW

```
Compute one value
```

Example statement:

C#

```
switch (status)
{
    case GiftStatus.Completed:
        LogCompletion();
        SendReceipt();
        break;
}
```

Expression:

C#

```
string label =
    status switch
    {
        GiftStatus.Completed
            =>
            "Completed",

        _
            =>
            "Other"
    };
```

Microsoft makes the same distinction: statements suit actions, while switch expressions suit choosing a result.

14. Relational Patterns

Relational patterns compare against constants using:

EXAMPLE / FLOW

```
<

<=

>

>=
```

Example:

```C#
string ClassifyAmount(
    decimal amount)
    =>
    amount switch
    {
        < 0m
            =>
            "Invalid",

        0m
            =>
            "Zero",

        < 100m
            =>
            "Small",

        < 1000m
            =>
            "Medium",

        _
            =>
            "Major"
    };
```

Range Example

```C#
bool validPercentage = percentage is >= 0m and <= 100m;
```

This reads naturally as:

EXAMPLE / FLOW

```
percentage is
at least 0
AND
at most 100
```

Important Relational Rule

The value on the right side of a relational pattern is normally a compile-time constant.

This is valid:

```C#
amount is >= 1000m
```

But when your threshold is a variable:

```C#
decimal threshold = GetThreshold();
```

prefer:

```C#
amount >= threshold
```

or use a `when` guard in a switch.

Microsoft specifically distinguishes constant relational patterns from comparisons aga

PATTERNS / C#

variables.

15. Logical Patterns

Patterns can be combined with:

EXAMPLE / FLOW

```
and

or

not
```

and

C#

```
bool isMidSize = gift.TotalAmount is >= 1000m and < 10_000m;
```

Both patterns must match.

or

C#

```
bool needsReview = gift.Status is GiftStatus.Cancelled or GiftStatus.Reversed;
```

Either pattern may match.

not

C#

```
if (
    gift.Status
    is not GiftStatus.Completed)
{
}
```

Combined Pattern

C#

```
bool valid = percentage is (>= 0m and <= 100m) or 999m;
```

16. Logical Pattern Precedence

C# pattern operators bind in this order:

EXAMPLE / FLOW

```
1. not

2. and

3. or
```

Therefore:

PATTERNS / C#

```
C#
value is not 0 and < 10 or 100
```

can be difficult to read.

Prefer parentheses:

```
C#
value is
    (not 0 and < 10)
    or
    100;
```

Microsoft recommends parentheses when mixing logical patterns where they make intent clearer.

Pattern Operators Are Not Boolean Short-Circuit Operators

Do not assume:

```
EXAMPLE / FLOW
and
```

behaves exactly like:

```
EXAMPLE / FLOW
&&
```

in terms of evaluation ordering.

The language specification doesn't guarantee the evaluation order of all nested property/deconstruction operations involved in pattern matching.

Therefore avoid pattern code whose correctness depends on:

```
EXAMPLE / FLOW
Property getter A runs before B

Deconstruct method has side effects

Pattern evaluation mutates state
```

Pattern-tested members should ideally behave like pure observations.

17. Parenthesized Patterns

Parentheses can group patterns:

```
C#
bool accepted = priority is (>= 1 and <= 3) or 9;
```

Without parentheses, precedence rules still exist.

But parentheses often make business logic easier to understand.

Fundraising Example


```C#
bool requiresApproval = gift.TotalAmount is (> 10_000m and <= 100_000m) or > 250_000m;
```

You may decide a normal Boolean expression is clearer:

```C#
bool requiresApproval =
    (
        gift.TotalAmount > 10_000m
        &&
        gift.TotalAmount <= 100_000m
    )
    ||
    gift.TotalAmount > 250_000m;
```

Use patterns when they make the **shape or ranges** clearer, not merely because pattern syntax exists.

18. Property Patterns

A property pattern inspects properties of an object.

Example:

```C#
if (
    gift is
    {
        Status:
            GiftStatus.Completed
    })
{
}
```

You can test several properties:

```C#
if (
    gift is
    {
        Status:
            GiftStatus.Completed,

        TotalAmount:
            >= 1000m
    })
{
}
```

This means:

```EXAMPLE / FLOW
gift is non-null

Status = Completed

TotalAmount >= 1000
```

Property patterns can contain nested patterns and type tests.

Property Pattern Automatically Excludes Null

This:

```C#
if (
    gift is
    {
        TotalAmount:
            > 0m
    })
{
}
```

doesn't match when:

```C#
gift is null
```

because there is no object whose property can satisfy the pattern.

19. Property Pattern with Variable Capture

```C#
if (
    gift is
    {
        TotalAmount:
            >= 1000m
            and
            var amount
    })
{
    Console.WriteLine(
        amount);
}
```

But don't force pattern syntax if this is clearer:

```C#
if (
    gift.TotalAmount >= 1000m)
{
    decimal amount = gift.TotalAmount;
}
```

Patterns are a readability tool, not a code-golf contest.

20. Nested Property Patterns

Suppose:

EXAMPLE / FLOW

```
Gift
    ↓
Campaign
    ↓
Name
```

You can write:

```C#
if (
    gift is
    {
        Campaign:
        {
            Name:
                "Annual Giving"
        }
    })
{
}
```

This says:

EXAMPLE / FLOW

```
Gift exists

Campaign exists

Campaign.Name is "Annual Giving"
```

Extended Property Pattern Syntax

Modern C# can often shorten nested access:

```C#
if (
    gift is
    {
        Campaign.Name:
            "Annual Giving"
    })
{
}
```

This is useful when the nested structure is simple.

For deeply nested models, the expanded form may still be easier to read.

21. Property Pattern + Relational Pattern

```C#
bool majorCompletedGift =
    gift is
    {
        Status:
            GiftStatus.Completed,

        TotalAmount:
            >= 10_000m
    };
```

This is a powerful pattern because multiple observations about one object are visually grouped together.

22. Property Pattern + Logical Pattern


```C#
bool actionable =
    gift is
    {
        Status:
            GiftStatus.Pending
            or
            GiftStatus.Failed
    };
```

Another:

```C#
bool normalAmount = gift is { TotalAmount: > 0m and <= 1_000_000m };
```

23. Empty Property Pattern { }

This:

```C#
gift is { }
```

matches any non-null object.

Therefore:

```C#
if (gift is { })
{
}
```

can be used as a non-null test.

However:

```C#
if (gift is not null)
```

is usually clearer when null checking is the only purpose.

24. Positional Patterns

Positional patterns match values obtained by deconstruction.

Suppose:

```C#
public readonly record struct MoneyRange(
    decimal Minimum,
    decimal Maximum);
```

Because a positional record supports deconstruction, you can write:

```C#
bool valid = range is (>= 0m, <= 100_000m);
```

Positional Pattern Mental Model

PATTERNS / C#

EXAMPLE / FLOW

```
Object
    ↓
Deconstruct(...)
    ↓
Position 1
Position 2
...
    ↓
Apply subpatterns
```

C# positional patterns can work through tuples, appropriate `Deconstruct` methods, and other supported positional mechanisms.

25. Custom Deconstruct

Example:

C#

```
public sealed class Gift
{
    public Gift(
        long giftId,
        decimal totalAmount)
    {
        GiftId = giftId;

        TotalAmount = totalAmount;
    }

    public long GiftId
    {
        get;
    }

    public decimal TotalAmount
    {
        get;
    }

    public void Deconstruct(
        out long giftId,
        out decimal totalAmount)
    {
        giftId = GiftId;

        totalAmount = TotalAmount;
    }
}
```

Now:

C#

```
if (
    gift is
    (
        > 0,
        >= 1000m
    ))
```

```
{
}
```

PATTERNS / C#

means:

EXAMPLE / FLOW

```
GiftId > 0

AND

TotalAmount >= 1000
```

Named Positional Elements

You can make the meaning clearer:

C#

```
if (
    gift is
    (
        giftId:
            > 0,

        totalAmount:
            >= 1000m
    ))
{
}
```

Use this when the positions alone would be ambiguous.

26. Property Pattern vs Positional Pattern

Property:

C#

```
gift is
{
    GiftId: > 0,
    TotalAmount: >= 1000m
}
```

Positional:

C#

```
gift is
(
    > 0,
    >= 1000m
)
```

Property patterns are often clearer for domain entities because business property names remain visible.

Positional patterns work especially well for:

EXAMPLE / FLOW

```
Records

Tuples
```

```
Coordinates
*
Small mathematical/value-like structures
```

PATTERNS / C#

27. Tuple Patterns

Tuples can be pattern matched directly.

Example:

C#

```
string GetTransition(
    GiftStatus oldStatus,
    GiftStatus newStatus)
    =>
    (
        oldStatus,
        newStatus
    )
    switch
    {
        (
            GiftStatus.Pending,
            GiftStatus.Completed
        )
            =>
            "Complete",

        (
            GiftStatus.Pending,
            GiftStatus.Cancelled
        )
            =>
            "Cancel",

        (
            GiftStatus.Completed,
            GiftStatus.Reversed
        )
            =>
            "Reverse",


        _
            =>
            "Invalid transition"
    };
```

Tuple Pattern Mental Model

EXAMPLE / FLOW

```
State A
+
State B
    ↓
One tuple input
    ↓
Match combinations
```

This is excellent for:

EXAMPLE / FLOW

```
State machines

Permission combinations

Transition rules

Two-dimensional decisions
```

PATTERNS / C#

Microsoft's pattern-matching tutorial demonstrates this style for state-transition logic.

Fundraising Workflow Example

C#

```
bool IsValidTransition(
    GiftStatus current,
    GiftStatus requested)
    =>
    (
        current,
        requested
    )
    is
    (
        GiftStatus.Pending,
        GiftStatus.Completed
    )
    or
    (
        GiftStatus.Pending,
        GiftStatus.Cancelled
    )
    or
    (
        GiftStatus.Completed,
        GiftStatus.Reversed
    );
```

This can be easier to audit than a large nested `if`.

28. List Patterns

List patterns match:

EXAMPLE / FLOW

```
Sequence length

Specific elements

Patterns on elements

Beginning/end structure
```

Example:

C#

```
int[] numbers = [1, 2, 3];

bool exact =
    numbers is
```

```
    [
        1,
        2,
        3
    ];
```

This checks both:

EXAMPLE / FLOW

```
Length = 3

Values = 1, 2, 3
```

C# list patterns describe the observable shape of supported sequence-like values.

Element Patterns

C#

```
bool valid =
    amounts is
    [
        > 0m,
        > 0m,
        > 0m
    ];
```

This requires exactly three positive values.

Discards Inside Lists

C#

```
bool startsWithGiftCode =
    parts is
    [
        "GIFT",
        _,
        _
    ];
```

Means:

EXAMPLE / FLOW

```
Exactly three elements

First is "GIFT"

Other two may contain anything
```

29. Slice Pattern . .

A slice pattern allows the middle/remainder of a sequence.

C#

```
bool startsCorrectly = parts is ["GIFT",..];
```

Meaning:

EXAMPLE / FLOW

```
First element must be "GIFT"

Any number of remaining elements allowed
```

Start and End

C#

```
bool valid =
    parts is
    [
        "GIFT",
        ..,
        "END"
    ];
```

The first and final markers are tested.

The middle may contain other elements.

Capturing a Slice

Depending on the supported input shape, a slice can also be captured:

C#

```
if (
    values is
    [
        var first,
        .. var middle,
        var last
    ])
{
}
```

Use list patterns when the **structure of the sequence itself** is part of the rule.

30. List Pattern vs LINQ

Suppose you want:

EXAMPLE / FLOW

```
Any Gift amount > 1000
```

Use LINQ:

C#

```
amounts.Any(
    amount =>
        amount > 1000m);
```

If you want:

EXAMPLE / FLOW

```
Exactly three elements

First is 100
```

```
Last is 500
```


a list pattern may better express the shape:

```C#
amounts is
[
    100m,
    _,
    500m
]
```

Use:

```EXAMPLE / FLOW
LINQ
    →
Query sequence contents

List patterns
    →
Match sequence shape
```

31. Switch Guards with `when`

Sometimes the pattern itself doesn't contain all the required information.

Use:

```C#
when
```

Example:

```C#
string Classify(
    Gift gift,
    decimal tenantThreshold)
    =>
    gift switch
    {
        {
            Status:
                GiftStatus.Completed
        }
            when gift.TotalAmount
                >=
                tenantThreshold
            =>
            "Major completed gift",

        {
            Status:
                GiftStatus.Completed
        }
            =>
            "Completed gift",


        _
            =>
```

```
            "Other"
    };
```

PATTERNS / C#

`when` is appropriate when the decision depends on a separate runtime condition or a Boolean expression that doesn't fit naturally into a pattern.

Pattern vs Guard

Prefer pattern:

```
C#
amount is >= 1000m
```

when the threshold is a constant.

Prefer guard:

```
C#
case Gift gift
    when gift.TotalAmount >= threshold:
```

when:

```
EXAMPLE / FLOW
threshold
```

is a runtime variable.

32. Arm Ordering

Switch-expression arms are tested in source order.

Example:

```
C#
string result =
    amount switch
    {
        >= 100m
            =>
            "100+",

        >= 1000m
            =>
            "1000+",


        _
            =>
            "Other"
    };
```

This is wrong.

Why?

Any value:

```
EXAMPLE / FLOW
>= 1000
```

already matches:

so the later arm can never run.

The compiler detects many such unreachable/subsumed patterns.

Correct Order

More specific first:

```C#
string result =
    amount switch
    {
        >= 1000m
            =>
            "1000+",

        >= 100m
            =>
            "100+",


        _
            =>
            "Other"
    };
```

General Rule

```
EXAMPLE / FLOW
Most specific patterns
        ↓
More general patterns
        ↓
Catch-all _
```

33. Exhaustiveness

A switch expression should normally cover every possible input.

Example:

```C#
string label =
    status switch
    {
        GiftStatus.Pending
            =>
            "Pending",

        GiftStatus.Completed
            =>
            "Completed",

        GiftStatus.Cancelled
            =>
            "Cancelled",


        _
```

```
            =>
            "Unknown"
    };
```

PATTERNS / C#

The final:

```
C#
_
```

guarantees a fallback.

Non-Exhaustive Switch Expression

Suppose:

```
C#
string label =
    status switch
    {
        GiftStatus.Pending
            =>
            "Pending"
    };
```

If no arm matches at runtime, a modern .NET switch expression throws:

```
EXAMPLE / FLOW
SwitchExpressionException
```

The compiler warns about many non-exhaustive cases.

Important List-Pattern Caveat

Microsoft notes that list patterns don't always produce non-exhaustiveness warnings for every possible unhandled list shape.

So don't assume:

```
EXAMPLE / FLOW
No compiler warning
=
Every possible list shape handled
```

34. Enum Exhaustiveness

Suppose:

```
C#
public enum GiftStatus
{
    Pending,
    Completed,
    Cancelled
}
```

You could write:

```C#
string label =
    status switch
    {
        GiftStatus.Pending
            =>
            "Pending",

        GiftStatus.Completed
            =>
            "Completed",

        GiftStatus.Cancelled
            =>
            "Cancelled",


        _
            =>
            throw new ArgumentOutOfRangeException(
                nameof(status),
                status,
                null)
    };
```

Why keep the default?

Because an enum variable can theoretically contain an underlying integral value not explicitly named by the enum.

For externally sourced data, defensive handling can be appropriate.

35. Pattern Matching Polymorphic Hierarchies

Suppose:

```C#
public abstract class DonationSource
{
}

public sealed class OnlineDonation :
    DonationSource
{
    public string TransactionId
    {
        get;
        init;
    } = "";
}

public sealed class CheckDonation :
    DonationSource
{
    public string CheckNumber
    {
        get;
        init;
    } = "";
}
```

Then:

```C#
string Describe(
    DonationSource source)
    =>
    source switch
    {
        OnlineDonation online
            =>
            $"Online {online.TransactionId}",

        CheckDonation check
            =>
            $"Check {check.CheckNumber}",


        _
            =>
            "Other"
    };
```

Pattern matching is excellent when code must **inspect** different known variants.

But Don't Replace Polymorphism Everywhere

Suppose every new donation subtype requires modifying:

```C#
Describe()

CalculateFee()

Validate()

Process()

Render()
```

and each method contains a large switch over subtypes.

That may indicate behavior belongs on the polymorphic objects themselves:

```C#
source.CalculateFee()
```

rather than repeated type switching.

Use:

```
EXAMPLE / FLOW
Pattern matching
```

when the caller legitimately needs to interpret variants.

Use:

```
EXAMPLE / FLOW
Polymorphism
```

when each type naturally owns its behavior.

36. Pattern Matching `object`

PATTERNS / C#

Pattern matching is especially useful when the compile-time type is broad.

C#

```
static string Format(
    object? value)
    =>
    value switch
    {
        null
            =>
            "(null)",

        decimal amount
            =>
            amount.ToString(
                "C"),

        DateTimeOffset timestamp
            =>
            timestamp.ToString(
                "O"),

        string text
            =>
            text,


        _
            =>
            value.ToString()
            ??
            string.Empty
    };
```

This is safer and clearer than:

EXAMPLE / FLOW

```
if type
cast
else if type
cast
...
```

37. Nullable Value Types

Suppose:

C#

```
decimal? amount = GetAmount();
```

You can write:

C#

```
if (
    amount is decimal value)
{
    Console.WriteLine(
        value);
}
```

This succeeds only when the nullable contains a value.

The variable:

```C#
value
```

is:

```EXAMPLE / FLOW
decimal
```

not:

```EXAMPLE / FLOW
decimal?
```

Pattern matching can therefore combine:

```EXAMPLE / FLOW
HasValue check
+
value extraction
```

38. Negative Type Pattern

```C#
if (
    value is not Gift)
{
    return;
}
```

Simple and readable.

If you need a typed local:

```C#
if (
    value is not Gift gift)
{
    return;
}

Process(
    gift);
```

After the guard exits on failure, the compiler can understand that the remaining path has a matched Gift.

This is a useful guard-clause style.

39. Pattern Matching and Guard Clauses

Old:

```C#
Gift? gift = value as Gift;

if (gift is null)
{
```

```
    throw new ArgumentException();
}
```

PATTERNS / C#

Modern:

```C#
if (
    value is not Gift gift)
{
    throw new ArgumentException(
        "Expected a Gift.",
        nameof(value));
}

Process(
    gift);
```

This often produces clearer flow.

40. Pattern Matching Validation

Example:

```C#
public static void ValidatePercentage(
    decimal percentage)
{
    if (
        percentage
        is < 0m or > 100m)
    {
        throw new ArgumentOutOfRangeException(
            nameof(percentage));
    }
}
```

This reads naturally:

```EXAMPLE / FLOW
percentage is
less than 0
OR
greater than 100
```

Fundraising Validation

```C#
public void Allocate(
    decimal amount)
{
    if (
        amount
        is <= 0m)
    {
        throw new ArgumentOutOfRangeException(
            nameof(amount));
    }

    // ...
}
```

Whether this is clearer than:

```C#
if (amount <= 0m)
```

is subjective.

Don't replace familiar comparisons when the pattern syntax adds no value.

41. Pattern Matching State Machine

Consider Gift state transitions:

```C#
public enum GiftStatus
{
    Draft,
    Pending,
    Completed,
    Cancelled,
    Reversed
}
```

Validation:

```C#
public static bool CanTransition(
    GiftStatus current,
    GiftStatus target)
    =>
    (
        current,
        target
    )
    switch
    {
        (
            GiftStatus.Draft,
            GiftStatus.Pending
        )
            =>
            true,

        (
            GiftStatus.Pending,
            GiftStatus.Completed
        )
            =>
            true,

        (
            GiftStatus.Pending,
            GiftStatus.Cancelled
        )
            =>
            true,

        (
            GiftStatus.Completed,
            GiftStatus.Reversed
        )
            =>
            true,
```

```
    _
        =>
        false
};
```

This is one of the strongest use cases for tuple patterns.

Flow

EXAMPLE / FLOW

```
(Current Status, Requested Status)
            ↓
        switch
            ↓
Known valid transition?
      ├─ Yes → true
      └─ No  → false
```

42. Pattern Matching Factory Logic

Example:

C#

```
public static IReceiptFormatter
    CreateFormatter(
        ReceiptType type)
    =>
    type switch
    {
        ReceiptType.Email
            =>
            new EmailReceiptFormatter(),

        ReceiptType.Print
            =>
            new PrintReceiptFormatter(),

        ReceiptType.None
            =>
            new NullReceiptFormatter(),


        _
            =>
            throw new ArgumentOutOfRangeException(
                nameof(type))
    };
```

A switch expression is often cleaner than:

EXAMPLE / FLOW

```
if / else if / else if
```

for fixed choices that produce one value.

43. Pattern Matching vs Dictionary Lookup

For simple key-to-value mappings:

```C#
Dictionary<GiftStatus, string>
```

might sometimes be better than a large switch.

Switch:

```C#
status switch
{
    GiftStatus.Pending
        =>
        "Pending",

    GiftStatus.Completed
        =>
        "Completed",


    _
        =>
        "Unknown"
};
```

Dictionary can fit when mappings are:

```EXAMPLE / FLOW
Data-driven

Configurable

Large

Frequently looked up
```

Pattern matching is best when logic itself is part of the program's decision structure.

44. Pattern Matching Performance

Pattern matching is a language feature.

The compiler lowers patterns into runtime operations appropriate to the pattern.

Examples may become combinations of:

```EXAMPLE / FLOW
Type tests

Comparisons

Property reads

Branches

Deconstruction calls

Index/length operations
```

There isn't one universal performance cost called:

```EXAMPLE / FLOW
"pattern matching overhead"
```

The cost depends on what your pattern actually does.

Type Pattern

```C#
value is Gift gift
```

typically involves runtime type checking and assignment.

Property Pattern

```C#
gift is
{
    Campaign.Name:
        "Annual"
}
```

requires accessing the relevant members.

Positional Pattern

```C#
gift is
(
    > 0,
    > 100m
)
```

may invoke:

```C#
Deconstruct
```

depending on the type.

List Pattern

List patterns may inspect:

```EXAMPLE / FLOW
Length / Count

Indexes

Slices
```

depending on the input.

Performance Rule

Prefer:

```EXAMPLE / FLOW
Clear pattern
```

first.

Optimize only when profiling shows this branch is hot.

PATTERNS / C#

45. Avoid Side Effects in Pattern-Tested Properties

The language specification states that the evaluation order of operations such as property accesses and `Deconstruct` calls inside pattern matching isn't generally specified.

Therefore this is poor design:

C#

```
public int Amount
{
    get
    {
        _counter++;

        return _amount;
    }
}
```

followed by complicated patterns involving `Amount`.

Property getters used by patterns should ideally behave as observations:

EXAMPLE / FLOW

```
No mutation

No external I/O

No surprising side effects
```

46. Pattern Matching and Security

Pattern matching is not authorization.

This:

C#

```
if (
    gift is
    {
        TenantId:
            var tenantId,

        Status:
            GiftStatus.Completed
    })
{
}
```

does not prove:

EXAMPLE / FLOW

```
Current caller owns tenantId
```

You still need:

EXAMPLE / FLOW

```
Authenticated user

Authorization rules

Tenant validation

Data-access filtering
```

PATTERNS / C#

Pattern matching only describes program state.

Do Not Trust Shape as Permission

Bad:

C#

```
if (
    request is
    {
        IsAdmin:
            true
    })
{
    DeleteEverything();
}
```

if:

EXAMPLE / FLOW

```
IsAdmin
```

came directly from client input.

Patterns help inspect data.

They don't establish trust.

Part 2 — Modern C# Around Pattern-Centric Code

C# has evolved toward code that is:

EXAMPLE / FLOW

```
More expression-oriented

More null-aware

More declarative

More concise without losing static typing
```

Many features have already been covered in earlier roadmap topics:

EXAMPLE / FLOW

```
Records

Nullable reference types

Collection expressions

Extension members
```

This section highlights the modern C# 14 features most useful alongside pattern-oriented code.

Microsoft currently documents C# 14 as the stable language release associated with .NET 10.

47. C# 14 Null-Conditional Assignment

Before C# 14:

```C#
if (
    gift.Campaign is not null)
{
    gift.Campaign.DisplayName = normalizedName;
}
```

C# 14 permits assignment through:

```C#
?.
```

Example:

```C#
gift.Campaign?.DisplayName = normalizedName;
```

The assignment occurs only if the left-side receiver is non-null.

Even more importantly, the right-hand expression is evaluated only when the receiver is non-null.

Example

```C#
campaign?.DisplayName = NormalizeName();
```

If:

```C#
campaign is null
```

then:

```C#
NormalizeName()
```

isn't called.

Compound Assignment

C# 14 also permits forms such as:

```C#
metrics?.GiftCount += 1;
```

But:

```C#
metrics?.GiftCount++;
```

is not allowed.

Null-conditional assignment supports assignment and compound assignment, but not `++` / `--`.

48. Null Pattern vs Null-Conditional Assignment

Use:

```C#
if (
    campaign is not null)
{
    campaign.Name = value;

    campaign.Status = status;

    Audit(
        campaign);
}
```

when multiple operations depend on non-null state.

Use:

```C#
campaign?.Name = value;
```

when there is one simple conditional assignment.

Don't compress large workflows into repeated:

```C#
?.
```

if a normal guard makes intent clearer.

49. C# 14 `field` Keyword

C# 14 adds a contextual:

```C#
field
```

keyword for property accessors.

Example:

```C#
public string Name
{
    get;
    set
    {
        ArgumentException.ThrowIfNullOrWhiteSpace(
            value);

        field = value.Trim();
```

```
    }
}
```


You no longer need an explicit backing field such as:

```
C#
private string _name;
```

for many validation/normalization cases.

Before

```
C#
private string _name = "";

public string Name
{
    get =>
        _name;

    set =>
        _name = value.Trim();
}
```

C# 14:

```
C#
public string Name
{
    get;

    set =>
        field = value.Trim();
}
```

When Useful

```
EXAMPLE / FLOW
Validation

Normalization

Lazy initialization

Property-change logic
```

But if the backing storage itself has domain meaning, an explicitly named field may still be clearer.

50. C# 14 nameof with Unbound Generic Types

C# 14 permits:

```
C#
nameof(
    List<>)
```

which evaluates to:

EXAMPLE / FLOW

```
"List"
```

PATTERNS / C#

Previously you commonly needed a closed constructed form such as:

C#

```
nameof(
    List<int>)
```

for this scenario.

Useful in:

EXAMPLE / FLOW

```
Diagnostics

Generic library code

Logging

Analyzers/framework infrastructure
```

Less important in normal business logic.

51. C# 14 Lambda Parameter Modifiers

Modern C# allows modifiers such as:

EXAMPLE / FLOW

```
ref

in

out

scoped

ref readonly
```

on simple lambda parameters without writing every explicit parameter type.

Example:

C#

```
delegate bool TryParse<T>(
    string text,
    out T result);

TryParse<int> parser =
    (
        text,
        out result
    )
    =>
    int.TryParse(
        text,
        out result);
```

C# 14 relaxed the requirement that modifiers force all parameter types to be written explicitly.

This matters mostly in:

EXAMPLE / FLOW

```
Advanced delegate APIs

Performance-sensitive libraries

Parsing APIs
```

PATTERNS / C#

rather than everyday Web API services.

52. Modern C# Design Principle

Modern syntax should reduce:

EXAMPLE / FLOW

```
Noise
```

not:

EXAMPLE / FLOW

```
Clarity
```

Compare:

C#

```
bool valid =
    gift is
    {
        Status:
            GiftStatus.Completed,

        TotalAmount:
            > 0m
    };
```

with an overcomplicated single expression containing:

EXAMPLE / FLOW

```
Nested property patterns

Multiple or branches

var patterns

when guards

Tuple nesting

List patterns
```

If it takes more time to decode than a normal `if`, simplify it.

When Pattern Matching Is Best

Pattern matching is especially effective for:

EXAMPLE / FLOW

```
Runtime type discrimination

Null checks
```

```
Range classification

Object shape inspection

State transitions

Small closed sets of cases

Nested structural conditions

Sequence shape matching

Value-returning branching
```

When `if` Is Better

Use an ordinary `if` when:

EXAMPLE / FLOW

```
There is one simple Boolean condition

Conditions depend on unrelated variables

Heavy method calls are involved

The pattern version becomes cryptic

Sequential imperative logic matters
```

When Polymorphism Is Better

Prefer polymorphism when:

EXAMPLE / FLOW

```
Behavior naturally belongs to each subtype

Many operations repeatedly switch on the same type hierarchy

Adding a subtype would require editing many switch statements
```

When a Dictionary Is Better

Prefer a lookup structure when:

EXAMPLE / FLOW

```
The logic is mostly static key → value data

Mappings are large

Mappings may be configured dynamically
```

Common Mistake 1 — Test Then Cast

Avoid:

C#

```
if (
    value is Gift)
```

```
{
    Gift gift = (Gift)value;
}
```

Prefer:

```C#
if (
    value is Gift gift)
{
}
```

Common Mistake 2 — Using as + Null When Declaration Pattern Is Clearer

Instead of:

```C#
Gift? gift = value as Gift;

if (
    gift is not null)
{
}
```

prefer:

```C#
if (
    value is Gift gift)
{
}
```

when you only need the typed value inside the matched flow.

Common Mistake 3 — Overusing Switch

This:

```C#
bool positive =
    amount switch
    {
        > 0m
            =>
            true,

        _
            =>
            false
    };
```

is more complicated than:

```C#
bool positive = amount > 0m;
```

Common Mistake 4 — Incorrect Arm Order

Wrong:

```
C#
amount switch
{
    >= 100m
        =>
        "Large",

    >= 1000m
        =>
        "Major",


    _
        =>
        "Other"
};
```

Put the more specific case first.

Common Mistake 5 — Missing Catch-All

If all runtime values aren't guaranteed to be covered:

```
C#
_
```

or explicit error handling is usually appropriate.

Common Mistake 6 — Assuming _ Is Always a Variable

In pattern context:

```
C#
_
```

is a discard.

It means:

```
EXAMPLE / FLOW
I don't care about this value.
```

Common Mistake 7 — Confusing `var` Pattern with `var` Local Declaration

This:

```
C#
var gift = GetGift();
```

is local type inference.

This:

```C#
value is var captured
```

is a pattern.

Different syntax contexts.

Common Mistake 8 — Assuming `var` Pattern Rejects Null

It does not.

A `var` pattern matches every value, including `null`.

Common Mistake 9 — Using `== null` When You Want True Null Identity

Prefer:

```C#
is null

is not null
```

when you explicitly mean actual null-state testing and want to avoid overloaded equality behavior.

Common Mistake 10 — Runtime Variable in Relational Pattern

This isn't what relational-pattern syntax is for:

```C#
decimal threshold = GetThreshold();

// amount is > threshold
```

Use:

```C#
amount > threshold
```

or a `when` guard.

Relational pattern operands are based on constants.

Common Mistake 11 — Treating `and` as `&&`

Pattern operators follow pattern rules.

Don't depend on side-effect sequencing as though:

```C#
and
```

were simply a textual replacement for:

```C#
&&
```

Common Mistake 12 — Side-Effecting Property Getters

Avoid:

EXAMPLE / FLOW

```
Pattern reads property
    ↓
Property mutates state
```

especially because the language doesn't promise a simple left-to-right ordering of every property/deconstruction operation in nested patterns.

Common Mistake 13 — Deep Property Pattern That Nobody Can Read

Bad:

C#

```
request is
{
    Constituent.Address.Country.Code:
        "US",

    Campaign.Parent.Owner.Department.Code:
        "DEV",

    Gift.Allocations.Count:
        > 3
}
```

Even if legal for the particular model, deeply compressed structural logic can become harder to maintain.

Break complex business rules into named concepts.

Common Mistake 14 — Replacing Polymorphism with Giant Type Switches

If every method contains:

C#

```
value switch
{
    TypeA ...
    TypeB ...
    TypeC ...
}
```

reconsider whether those types should own the behavior.

Common Mistake 15 — Treating Patterns as Security

A value matching:

C#

```
{ TenantId: 10 }
```

does not mean the current user is authorized for Tenant 10.

Patterns inspect data.

Authorization establishes permission.

Best Practices

Use declaration patterns to combine:

EXAMPLE / FLOW

```
Type test
+
safe typed variable
```

Prefer:

C#

```
is null

is not null
```

for clear null-state checks.

Use relational patterns for readable constant ranges.

Use:

C#

```
and

or

not
```

when they make the value rules easier to understand.

Use parentheses when logical-pattern precedence isn't immediately obvious.

Prefer property patterns when business rules depend on several properties of the same object.

Prefer positional patterns mainly for compact value-like types, tuples, or meaningful deconstruction.

Use tuple patterns for state transitions and multi-value decisions.

Use list patterns when sequence **shape** matters.

Use LINQ when sequence **querying** matters.

Keep switch arms:

EXAMPLE / FLOW

```
Specific → general
```

Use:

C#

```
_
```

or an explicit throw when a fallback is required.

Use switch expressions to produce values.

Use switch statements when branches contain multi-step side effects.

Keep pattern-tested properties free of surprising side effects.

Don't force pattern syntax onto a simple Boolean condition.

Use polymorphism when subtype-specific behavior belongs to the subtype.

Use C# 14 syntax where it removes noise without hiding behavior.

Practical Exercise 1 — Type Pattern

Given:

```C#
object value = gift;
```

replace:

```C#
if (
    value is Gift)
{
    Gift casted = (Gift)value;
}
```

with:

```C#
if (
    value is Gift casted)
{
}
```

Practical Exercise 2 — Null Pattern

```C#
Gift? gift = FindGift();

if (
    gift is null)
{
    return;
}

Console.WriteLine(
    gift.TotalAmount);
```

The compiler understands that `gift` is non-null after the guard.

Practical Exercise 3 — Range Pattern

Validate allocation percentage:

```C#
bool valid = percentage is >= 0m and <= 100m;
```

Practical Exercise 4 — Property Pattern

```C#
bool majorCompleted =
    gift is
    {
        Status:
            GiftStatus.Completed,
```

```
    TotalAmount:
        >= 10_000m
};
```

Practical Exercise 5 — Nested Property Pattern

```C#
bool annualCampaignGift =
    gift is
    {
        Campaign:
        {
            Name:
                "Annual Giving"
        }
    };
```

Practical Exercise 6 — Tuple State Transition

```C#
bool canTransition =
    (
        current,
        requested
    )
    is
    (
        GiftStatus.Pending,
        GiftStatus.Completed
    )
    or
    (
        GiftStatus.Pending,
        GiftStatus.Cancelled
    );
```

Practical Exercise 7 — Switch Expression

```C#
string label =
    gift.Status switch
    {
        GiftStatus.Pending
            =>
            "Pending",

        GiftStatus.Completed
            =>
            "Completed",

        GiftStatus.Cancelled
            =>
            "Cancelled",


        _
            =>
```

```
            "Unknown"
    };
```

Practical Exercise 8 — when Guard

```C#
string category =
    gift switch
    {
        {
            Status:
                GiftStatus.Completed
        }
            when gift.TotalAmount
                >=
                tenantMajorGiftThreshold
            =>
            "Major",

        {
            Status:
                GiftStatus.Completed
        }
            =>
            "Standard",


        _
            =>
            "Other"
    };
```

Practical Exercise 9 — List Pattern

```C#
string[] parts = ["GIFT", "2026", "1001"];

bool valid =
    parts is
    [
        "GIFT",
        _,
        _
    ];
```

Practical Exercise 10 — Slice Pattern

```C#
bool valid =
    parts is
    [
        "GIFT",
        ..,
        "END"
    ];
```

Practical Exercise 11 — Polymorphic Switch

```C#
string Describe(
    object value)
    =>
    value switch
    {
        Gift gift
            =>
            $"Gift {gift.GiftId}",

        Campaign campaign
            =>
            campaign.Name,

        Fund fund
            =>
            fund.Name,


        _
            =>
            "Unknown"
    };
```

Practical Exercise 12 — C# 14 Null-Conditional Assignment

Before:

```C#
if (
    campaign is not null)
{
    campaign.DisplayName = normalizedName;
}
```

C# 14:

```C#
campaign?.DisplayName = normalizedName;
```

The right-hand side is evaluated only when the receiver exists.

Exactly 25 Interview Questions

1. What is pattern matching in C#?

Pattern matching lets code test an input against a type, value, or structural shape and optionally extract information when the pattern matches.

Patterns are primarily used with:

```C#
is

switch statement

switch expression
```

2. What is a declaration pattern?

A declaration pattern simultaneously tests runtime type compatibility and declares a typed variable.

```C#
if (
    value is Gift gift)
{
}
```

If the pattern succeeds, `gift` is definitely assigned and can be used without a cast.

3. What is the difference between a type pattern and a declaration pattern?

Type pattern:

```C#
value is Gift
```

only tests the type.

Declaration pattern:

```C#
value is Gift gift
```

tests the type and captures the value in a new variable.

4. Why is `is null` often preferred for explicit null testing?

Because:

```C#
value is null
```

tests actual null state without invoking overloaded `==` behavior.

Likewise:

```C#
value is not null
```

provides a clear non-null test.

5. What is a constant pattern?

A constant pattern tests whether an input equals a particular constant value.

Example:

```C#
status is
    GiftStatus.Completed
```

`null` is also commonly used as a constant pattern.

6. What is the `var` pattern?

The `var` pattern captures the evaluated input without testing its type or value.

```C#
expression is var result
```

It always matches, including when the value is `null`.

7. What is the discard pattern?

The:

```C#
_
```

pattern matches any value and intentionally ignores it.

It is commonly used as the final arm of a switch expression:

```C#
_
    =>
    defaultResult
```

8. What are relational patterns?

Relational patterns compare an input against a constant with:

```EXAMPLE / FLOW
<

<=

>

>=
```

Example:

```C#
amount is
    >= 1000m and < 10_000m
```

The relational-pattern operand is a constant; runtime-variable comparisons are better expressed with ordinary Boolean expressions or guards.

9. What are logical patterns?

Logical patterns combine or negate other patterns:

```EXAMPLE / FLOW
and

or

not
```

For example:

```C#
status is
    GiftStatus.Pending
```

```
    or
    GiftStatus.Failed
```

10. What is the precedence of pattern operators?

The binding order is:

EXAMPLE / FLOW

```
not

and

or
```

Use parentheses whenever a combined pattern could be misunderstood.

11. What is a property pattern?

A property pattern tests the observable properties of an object.

C#

```
gift is
{
    Status:
        GiftStatus.Completed,

    TotalAmount:
        >= 1000m
}
```

The pattern doesn't match a null object. Property patterns can contain nested patterns.

12. What is a positional pattern?

A positional pattern matches values extracted by positional decomposition, commonly using tuples or a type's `Deconstruct` method.

Example:

C#

```
point is
(
    0,
    > 0
)
```

13. What is a tuple pattern?

A tuple pattern matches several values together.

Example:

C#

```
(
    currentStatus,
    targetStatus
)
switch
{
    (
        GiftStatus.Pending,
```

```
        GiftStatus.Completed
    )
        =>
        true,

    _
        =>
        false
};
```

Tuple patterns are particularly useful for state transitions and multi-variable decisions.

14. What is a list pattern?

A list pattern matches the observable shape and element values of a sequence.

Example:

```C#
values is
[
    1,
    2,
    3
]
```

requires exactly those three positions to match.

15. What does .. mean inside a list pattern?

It is a slice pattern representing zero or more remaining sequence elements.

Example:

```C#
values is
[
    1,
    ..,
    5
]
```

requires the first element to match `1`, the last to match `5`, and allows any supported middle sequence.

16. What is the difference between a switch statement and a switch expression?

A switch statement is usually better for executing statements and side effects.

A switch expression is designed to select and return one result:

```C#
string label =
    status switch
    {
        ...
    };
```

17. What is a `when` guard?

A `when` guard adds a Boolean condition after a matching switch pattern.

```C#
case Gift gift
    when gift.TotalAmount >= threshold:
```

It is especially useful when the condition depends on runtime variables or logic outside the pattern input.

18. How does switch-arm ordering work?

Switch-expression arms are evaluated in textual order.

More-specific patterns should therefore appear before more-general patterns.

The compiler reports many cases where a later pattern is completely subsumed by an earlier one.

19. What happens when no switch-expression arm matches?

For a modern .NET switch expression, runtime execution throws:

EXAMPLE / FLOW

```
SwitchExpressionException
```

The compiler warns about many non-exhaustive switch expressions, although some pattern categories such as list patterns have limitations in exhaustiveness analysis.

20. Should pattern matching replace polymorphism?

No.

Pattern matching is good when a caller legitimately needs to inspect variants.

Polymorphism is generally better when behavior naturally belongs to each subtype and many callers would otherwise repeat large type switches.

21. Should pattern matching replace every `if` statement?

No.

This:

```C#
amount > 0m
```

is often clearer than:

```C#
amount is > 0m
```

Patterns are most useful when they clearly express:

EXAMPLE / FLOW

```
Types

Shapes

Ranges

Variants

Combinations
```

rather than merely replacing familiar operators.

22. Why should property getters used by patterns avoid side effects?

PATTERNS / C#

The language doesn't specify a simple fixed evaluation order for every property access, `Deconstruct` call, or related operation involved in nested pattern matching.

Therefore pattern correctness should not depend on those operations occurring in a particular order.

23. What C# 14 null-handling improvement is relevant to modern branching code?

C# 14 allows:

```
C#
customer?.Order = currentOrder;
```

and null-conditional compound assignments.

The right side is evaluated only when the null-conditional receiver isn't null.

24. What is the C# 14 `field` keyword?

Inside property accessors, `field` refers to the compiler-generated backing storage for that property.

Example:

```
C#
public string Name
{
    get;

    set =>
        field = value.Trim();
}
```

It reduces boilerplate for properties requiring custom accessor logic.

25. What is the most important design rule for modern pattern matching?

Use patterns when they make the program's valid states and structural conditions **more obvious**.

A good pattern:

```
EXAMPLE / FLOW
reveals intent
```

A bad pattern:

```
EXAMPLE / FLOW
compresses logic until nobody can read it
```

Pattern matching should improve clarity, type safety, and state modeling—not merely reduce line count.

Quick Revision

Feature	Purpose
`is`	Apply a pattern
Type pattern	Test runtime type
Declaration pattern	Test type + capture

Feature	Purpose
Constant pattern	Match specific value
`is null`	Actual null test
`is not null`	Non-null test
`var` pattern	Capture any value
`_`	Discard / catch-all
`<`, `>`, `<=`, `>=`	Relational patterns
`and`	Both patterns
`or`	Either pattern
`not`	Negated pattern
Property pattern	Match object properties
Positional pattern	Match deconstructed values
Tuple pattern	Match multiple values together
List pattern	Match sequence shape
`..`	List slice pattern
`when`	Additional Boolean guard
Switch statement	Action-oriented branching
Switch expression	Value-oriented branching
C# 14 `?. =`	Conditional assignment
C# 14 `field`	Property backing storage
`nameof(List<>)`	C# 14 unbound generic `nameof`

PATTERNS / C#

Complete Fundraising Example

Consider a Gift workflow where behavior depends on:

EXAMPLE / FLOW

```
Gift status

Gift amount

Campaign presence

Allocation state
```

We can model classification using patterns.

C#

```
public enum GiftStatus
{
    Draft,
    Pending,
    Completed,
    Cancelled,
    Reversed
```

```
}

public sealed class Campaign
{
    public Campaign(
        long campaignId,
        string name)
    {
        CampaignId = campaignId;

        Name = name;
    }

    public long CampaignId
    {
        get;
    }

    public string Name
    {
        get;
    }
}

public sealed class GiftAllocation
{
    public GiftAllocation(
        long fundId,
        decimal amount)
    {
        FundId = fundId;

        Amount = amount;
    }

    public long FundId
    {
        get;
    }

    public decimal Amount
    {
        get;
    }
}

public sealed class Gift
{
    public Gift(
        long giftId,
        decimal totalAmount,
        GiftStatus status,
        Campaign? campaign)
    {
        GiftId = giftId;

        TotalAmount = totalAmount;

        Status = status;

        Campaign = campaign;
    }

    public long GiftId
    {
```

```
        get;
    }

    public decimal TotalAmount
    {
        get;
    }

    public GiftStatus Status
    {
        get;
    }

    public Campaign? Campaign
    {
        get;
    }

    public List<GiftAllocation>
        Allocations
    {
        get;
    } = [];
}
```

Classification Function

C#

```
public static string ClassifyGift(
    Gift? gift)
    =>
    gift switch
    {
        null
            =>
            "No gift",

        {
            Status:
                GiftStatus.Cancelled
                or
                GiftStatus.Reversed
        }
            =>
            "Inactive gift",

        {
            Status:
                GiftStatus.Pending,

            TotalAmount:
                >= 10_000m
        }
            =>
            "Major pending gift",

        {
            Status:
                GiftStatus.Completed,

            Campaign:
            {
                Name:
```

```
                    "Annual Giving"
            },

            TotalAmount:
                >= 10_000m
        }
            =>
            "Major Annual Giving gift",

        {
            Status:
                GiftStatus.Completed,

            TotalAmount:
                > 0m
        }
            =>
            "Completed gift",


        _
            =>
            "Other gift"
    };
```

Flow

EXAMPLE / FLOW

```
Gift?
    ↓
null?
    ├─ Yes → "No gift"
    └─ No
        ↓
Cancelled/Reversed?
    ├─ Yes → "Inactive gift"
    └─ No
        ↓
Pending + >= 10,000?
    ├─ Yes → "Major pending gift"
    └─ No
        ↓
Completed + Annual Giving
+ >= 10,000?
    ├─ Yes → "Major Annual Giving gift"
    └─ No
        ↓
Completed + positive?
    ├─ Yes → "Completed gift"
    └─ No
        ↓
"Other gift"
```

This is a good example of patterns because the function is fundamentally:

EXAMPLE / FLOW

```
Classify one value
based on its shape/state
```

Allocation-State Pattern

PATTERNS / C#

C#

```
public static string GetAllocationState(
    Gift gift)
{
    decimal allocated = gift.Allocations.Sum(allocation => allocation.Amount);

    return (
        gift.TotalAmount,
        allocated
    )
    switch
    {
        (
            <= 0m,
            _
        )
            =>
            "Invalid gift",

        (
            var total,
            var current
        )
            when current > total
            =>
            "Overallocated",

        (
            var total,
            var current
        )
            when current == total
            =>
            "Fully allocated",

        (
            _,
            0m
        )
            =>
            "Unallocated",

        _
            =>
            "Partially allocated"
    };
}
```

This uses:

EXAMPLE / FLOW

```
Tuple pattern

Relational pattern

var pattern

when guard

discard pattern
```

in one realistic example.

State Transition Example

C#

```
public static bool CanTransition(
    GiftStatus current,
    GiftStatus requested)
    =>
    (
        current,
        requested
    )
    switch
    {
        (
            GiftStatus.Draft,
            GiftStatus.Pending
        )
            =>
            true,

        (
            GiftStatus.Pending,
            GiftStatus.Completed
        )
            =>
            true,

        (
            GiftStatus.Pending,
            GiftStatus.Cancelled
        )
            =>
            true,

        (
            GiftStatus.Completed,
            GiftStatus.Reversed
        )
            =>
            true,

        _
            =>
            false
    };
```

This is often much easier to verify than deeply nested branching.

Pattern Selection Decision Flow

EXAMPLE / FLOW

```
Need to test runtime type?
    →
Type / declaration pattern
```

EXAMPLE / FLOW

Need actual null check?
→
is null / is not null

EXAMPLE / FLOW

Need one exact value?
→
Constant pattern

EXAMPLE / FLOW

Need numeric range?
→
Relational + logical pattern

EXAMPLE / FLOW

Need several properties?
→
Property pattern

EXAMPLE / FLOW

Need deconstructed positions?
→
Positional pattern

EXAMPLE / FLOW

Need multiple independent values together?
→
Tuple pattern

EXAMPLE / FLOW

Need sequence shape?
→
List / slice pattern

EXAMPLE / FLOW

Need runtime variable or extra condition?
→
when guard / Boolean expression

EXAMPLE / FLOW

Need result value from branches?
→
switch expression

EXAMPLE / FLOW

Need branch actions?
→
switch statement / if

Pattern Matching vs Other C# Tools

EXAMPLE / FLOW

```
Simple Boolean rule
    →
if
```

EXAMPLE / FLOW

```
Type/value/shape classification
    →
pattern matching
```

EXAMPLE / FLOW

```
Behavior owned by subtype
    →
polymorphism
```

EXAMPLE / FLOW

```
Collection filtering/transformation
    →
LINQ
```

EXAMPLE / FLOW

```
Static key → value table
    →
Dictionary
```

EXAMPLE / FLOW

```
Cross-object business workflow
    →
services / explicit orchestration
```

Final End-to-End Concept Flow

EXAMPLE / FLOW

```
Input value
    ↓
Evaluate once
    ↓
Apply pattern
    ↓
Does pattern match?
    ├─ No
    │    ↓
    │ Try next pattern
    │
    └─ Yes
         ↓
       Capture values if requested
         ↓
       Evaluate when guard if present
         ↓
       Execute statement
       or produce expression result
```

Pattern categories:

EXAMPLE / FLOW

```
Type
    →
"What kind of value is this?"
```

EXAMPLE / FLOW

```
Constant
    →
"Is it this exact value?"
```

EXAMPLE / FLOW

```
Relational
    →
"Is it within this range?"
```

EXAMPLE / FLOW

```
Logical
    →
"Which combinations are valid?"
```

EXAMPLE / FLOW

```
Property
    →
"What shape/state does this object have?"
```

EXAMPLE / FLOW

```
Positional
    →
"What values does it deconstruct into?"
```

EXAMPLE / FLOW

```
Tuple
    →
"What combination of several values exists?"
```

EXAMPLE / FLOW

```
List
    →
"What shape does this sequence have?"
```

The core principle is:

> **Pattern matching should make valid states visible. Use it to describe types, values, ranges, object shapes, sequence shapes, and state combinations directly. Prefer switch expressions for value-producing decisions, tuple patterns for multi-state transitions, property patterns for object-state classification, and declaration patterns instead of manual type tests and casts. Modern C# syntax should remove accidental boilerplate without hiding the underlying business logic.**

For the Fundraising Platform, particularly useful pattern-matching scenarios include:

EXAMPLE / FLOW

```
Gift status transitions
```

```
Gift amount classification
Payment-source discrimination
Optional Campaign handling
Allocation-state classification
API/request validation
Mapping domain variants to responses
Interpreting small structured inputs
```

Next roadmap topic: Topic 27 — `async`, `await`, and Task-Based Programming.